



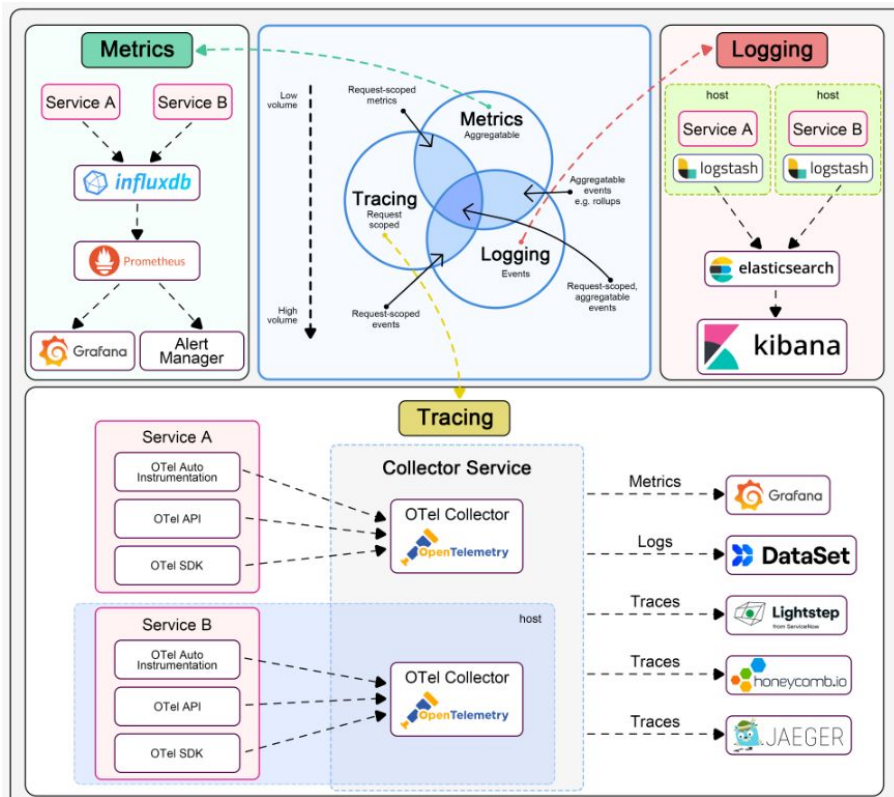
НАЦИОНАЛЬНЫЙ ИССЛЕДОВАТЕЛЬСКИЙ
УНИВЕРСИТЕТ

Системный дизайн современных приложений

Занятие №8
Итоги



Логи, метрики, трейсы, +1



Логи

- + Что случилось?
- + Нужна централизованная агрегация
- + ELK, Grafana + Loki, Vector, в качестве пушеров всякие filebeat, fluentbit

Метрики

- + Насколько плохо, как давно?
- + Числовые измерения
- + RED, 4 gold signals
- + Prometheus + Grafana, VictoriaMetrics

Трейсы

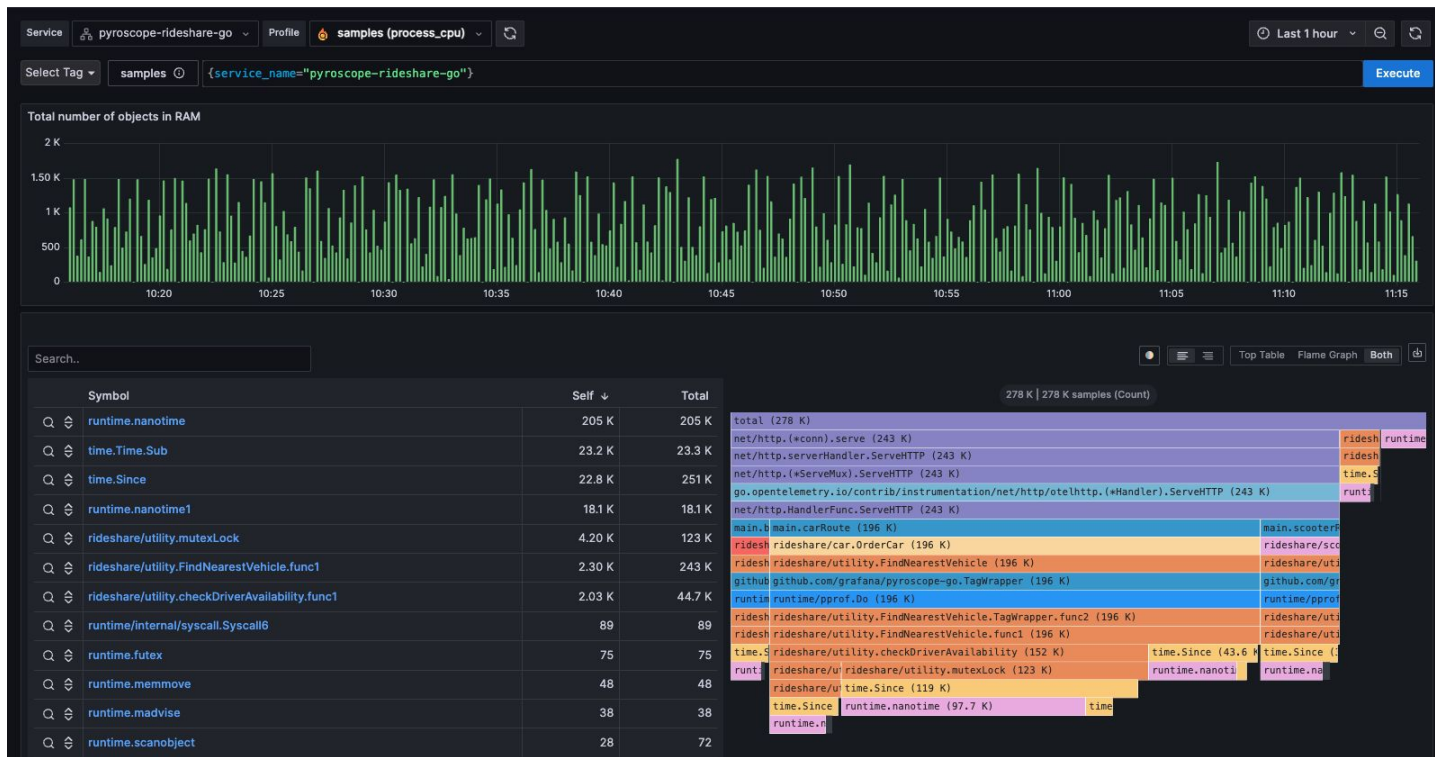
- + Где по системе оборвался/затупил запрос?
- + Сервисы создают span с общим trace_id
- + Jaeger, Zipkin

Профилирование

- + Где в коде?
- + Continuous Profiling - агент постоянно снимает профиль с каждого узла, данные централизуются
- + Pyroscope



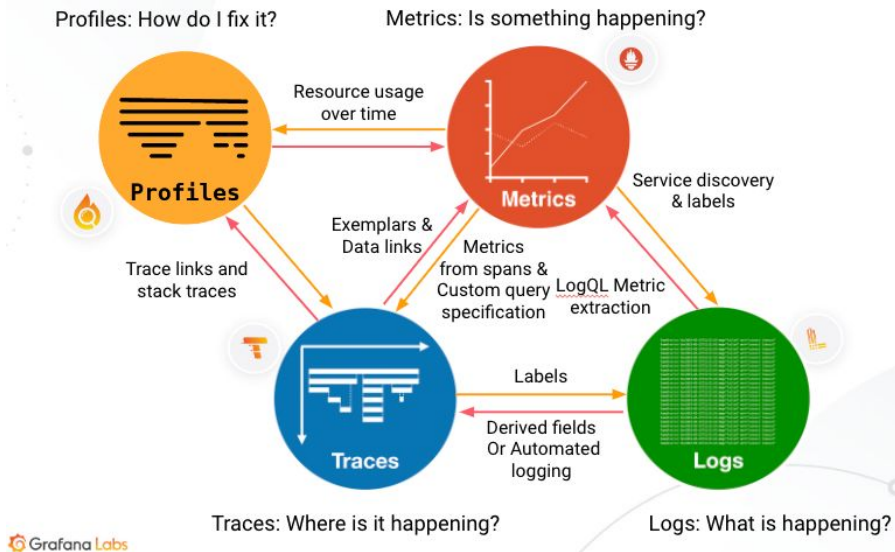
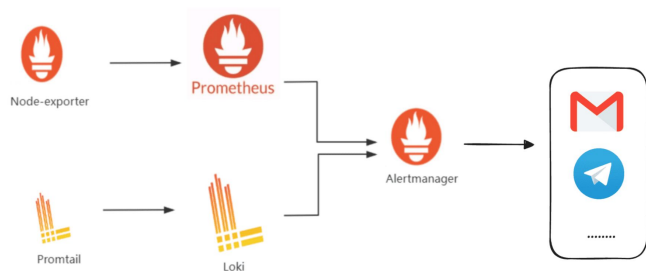
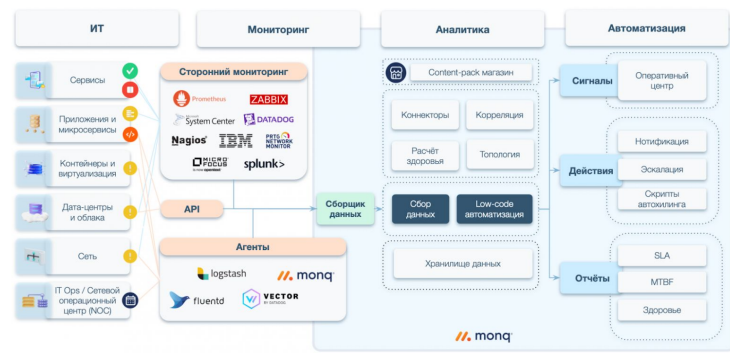
Continuous Profiling



Комбинация инструментов

Что?

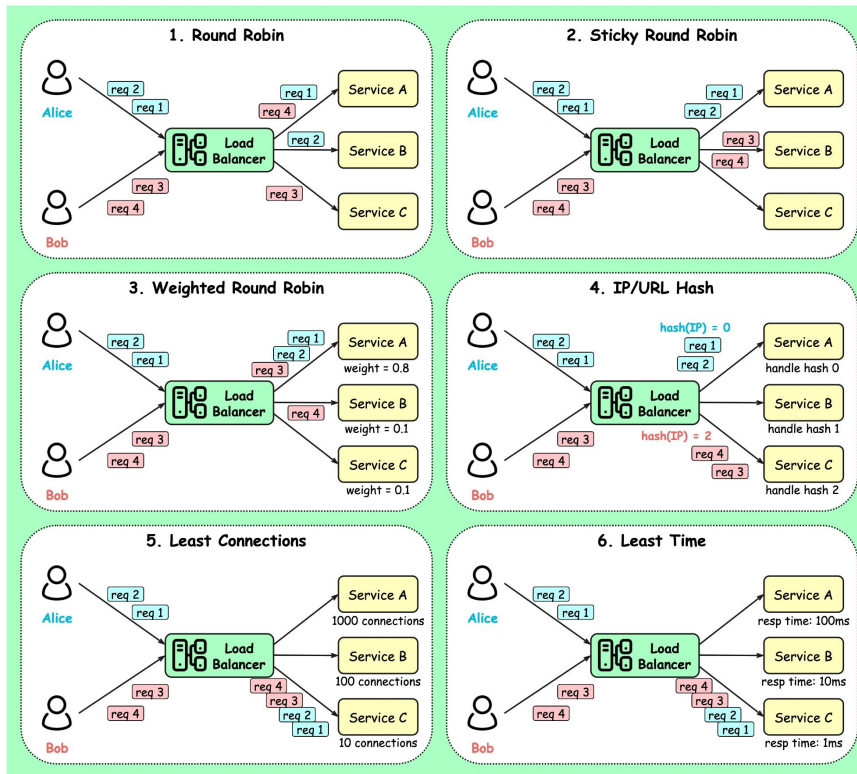
Зонтичный мониторинг: системы мониторинга систем мониторинга



Алгоритмы балансировки

Какие есть?

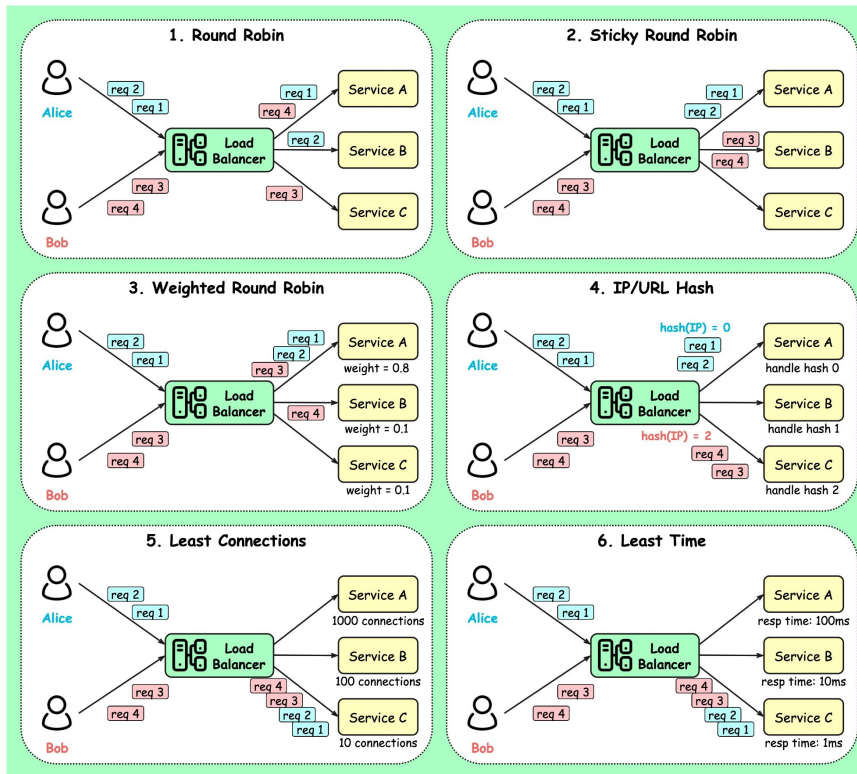
- Round Robin: когда запросы гомогенны и сервера идентичны. Ломается, когда запросы разные по весу: один запрос выполняется за 1 мс, другой за 5 секунд
- WRR: расширение для разнородного железа. Если один сервер вдвое мощнее, он получает вдвое больше запросов. Но по-прежнему не учитывает текущую загрузку
- Leastconn: отправляет новый запрос на сервер с наименьшим числом активных соединений. Ограничение: число соединений не всегда равно нагрузке
- Sticky: один и тот же клиент всегда попадает на один и тот же сервер. Неотказоустойчивый
- Least time / Least Response time: отправляет запрос на сервер с наименьшим средним временем ответа. Более точная мера нагрузки, чем число соединений. Требуется постоянное измерение - балансировщик следит за P99 каждого сервера и актуализирует рейтинг
- Hash: запрос всегда идет на один и тот же сервер, определяемый хэшем ключа



Алгоритмы балансировки

Какие есть?

- Round Robin: когда запросы гомогенны и сервера идентичны. Ломается, когда запросы разные по весу: один запрос выполняется за 1 мс, другой за 5 секунд
- WRR: расширение для разнородного железа. Если один сервер вдвое мощнее, он получает вдвое больше запросов. Но по-прежнему не учитывает текущую загрузку
- Leastconn: отправляет новый запрос на сервер с наименьшим числом активных соединений. Ограничение: число соединений не всегда равно нагрузке
- Sticky: один и тот же клиент всегда попадает на один и тот же сервер. Неотказоустойчивый
- Least time / Least Response time: отправляет запрос на сервер с наименьшим средним временем ответа. Более точная мера нагрузки, чем число соединений. Требуется постоянное измерение - балансировщик следит за P99 каждого сервера и актуализирует рейтинг
- Hash: запрос всегда идет на один и тот же сервер, определяемый хэшем ключа





Кэширование

Что?

Способ хранения данных как можно ближе к месту их использования

There are only two hard things in Computer Science: cache invalidation and naming things

Если можно обойтись без кэширования, оно не нужно

- + Кэширование не бесплатное
- + Не обязан становится сердцем системы
- + Нужно решить **что** кэшировать, **когда** инвалидировать и **что делать** если кэш упал
- + Если запрос к источнику данных уже быстрый и нагрузка не критичная - кэш добавит сложности без пользы

Temporal Locality

- + Более вероятно, что новые обращения будут происходить к недавним данным
- + Spatial Locality - если обратились к данным, вероятно скоро обратятся к соседним
- + Кэш с TTL в 5-10 минут ловит весь пиковый трафик

Принцип Парето

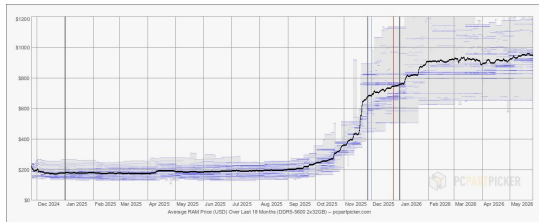
- + 80/20 - большая часть нагрузки на малой части данных
- + Нет смысла кэшировать все, только горячее. Задача аналитики: найти эти 5% запросов, которые генерируют 95% нагрузки

Кэш - чаще всего kv-хранилище

- + key-value дает $O(1)$ на чтение и запись через хэш-таблицу
- + Дай мне значение по ключу

Быстродействующая память (RAM/ОЗУ)

- + Разница в 100-1000 раз между обращением ОЗУ/диск

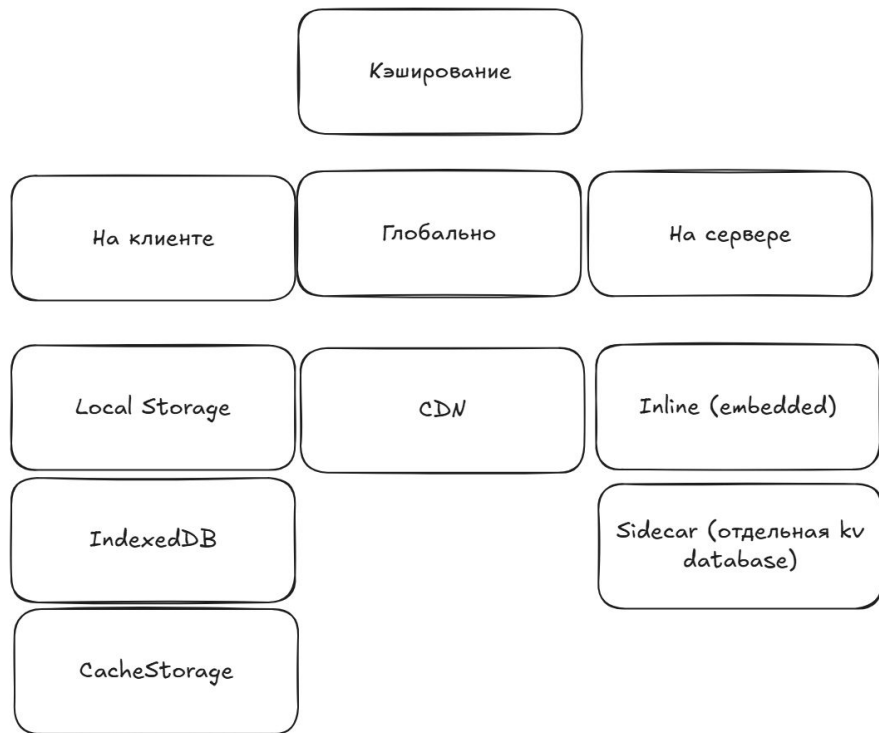


TTL

- + Каждая запись в кэше имеет TTL - “время жизни”, по истечении которого она удаляется
- + Когда TTL истекает у записи, которую запрашивают тысячи пользователей одновременно — все они одновременно идут в БД



Типы кэширования





Инвалидация кэша

Стоит всегда держать в голове

Рано или поздно данные в исходной БД изменятся и кэшированные данные придут в негодность

1. **Сквозная запись (write-through)**: данные сразу записываются и в источник, и в кэш

Плюсы: гарантированная консистентность, минимизация потерь

Минусы: запись становится даже медленнее из-за двух операций

2. **Запись в обход (cache aside)**: данные записываются напрямую в источник

Плюсы: не нагружаем кэш записью не востребуемых данных

Минусы: для недавних данных считывать нечего, идем в источник

3. **Реверсивная запись**: данные сначала записываются в кэш

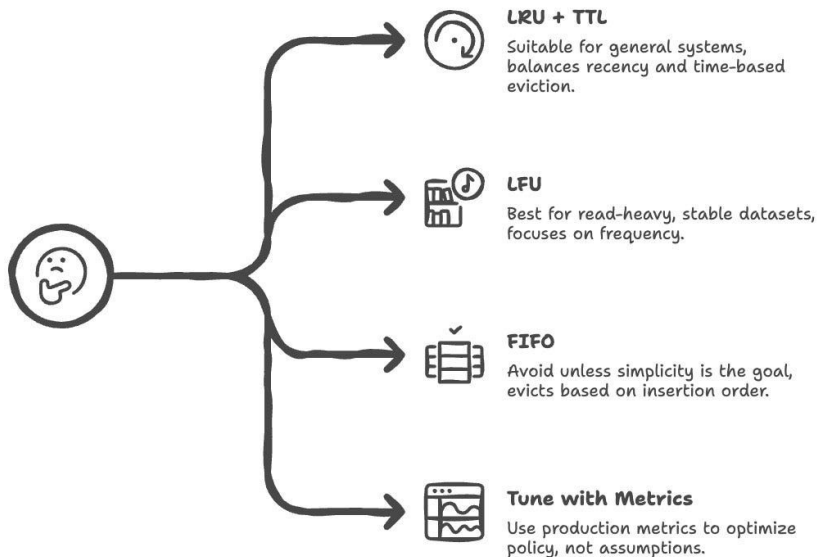
Плюсы: низкие задержки и высокая пропускная способность на запись

Минусы: можно потерять недавние данные, не продублированные в источник



Вытеснение данных

1. **First In First Out (FIFO)**: удаляем самые давние записи несмотря на их популярность
2. **Last In First Out (LIFO)**: удаляем самые свежие записи несмотря на их популярность
3. **Least Recently Used (LRU)**: удаляем записи, не использованные дольше всего
4. **Most Recently Used (MRU)**: удаляем самые недавние из использованных записей
5. **Least Frequently Used (LFU)**: ведем счет обращений, удаляем самые непопулярные
6. **Random Replacement (RR)**: удаляем случайно выбранные записи



Кэширование: целесообразность

Да	Возможно	Нет
Частые чтения, редкие записи	Частые чтения, частые записи	Редкие чтения, частые записи
Горячие данные (Паретто)	Очень большие объемы данных	Данные не могут быть неактуальными
Статика		Мало RPS
Результаты трудоемких вычислений		

Высокая доступность

How much data can you afford to recreate or lose?

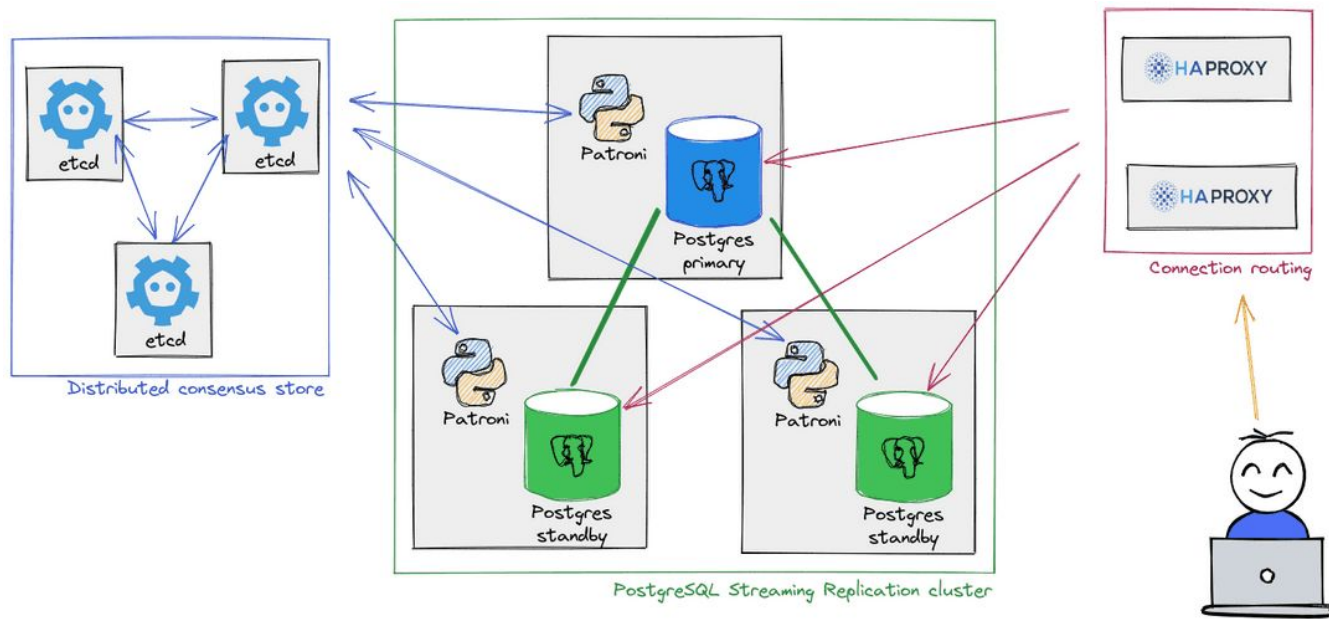
RPO vs RTO

How quickly must you recover?
What is the cost of downtime?

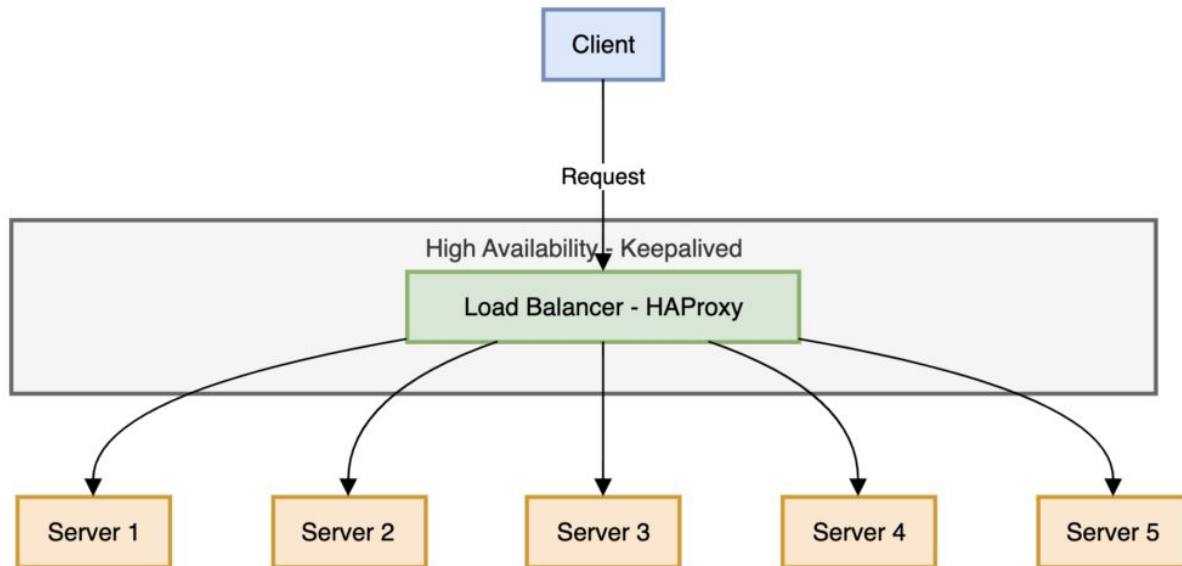


1. Ничего не делаем
2. Делаем бэкапы
3. Делаем репликацию
4. Делаем автоматическое переключение
5. Делаем устойчивость к отказам

HA PostgreSQL



Keepalived



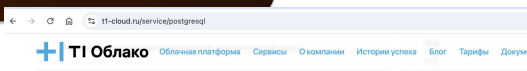


Как развернуть высокодоступную БД?

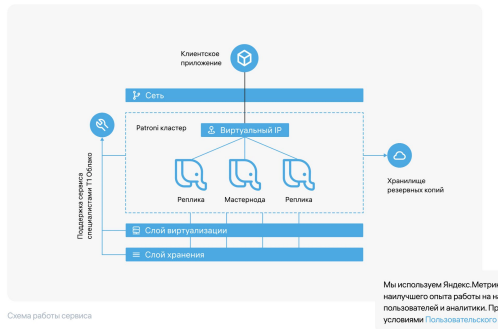
A combination of **both** worlds:

Automation of managed services with the flexibility of self-hosted infrastructure

	Managed	DevOps	Autobase
Control	✗	⚠	✓
Cost	✗	⚠	✓
Complexity	✓	✗	✓



Надежная архитектура



Зоны ответственности

Процесс	Пользователь	Т1 Облако
Установка и настройка ОС		✓
Установка и оптимизация параметров СУБД		✓
Репликация отказоустойчивого кластера		✓
Мониторинг состояния СУБД		✓
Обеспечение ресурсов для масштабирования		✓
Предоставление оборудования для СУБД		✓
Осуществление резервного копирования		✓
Обеспечение технической поддержки		✓
Подключение к БД и корректные выборки данных	✓	



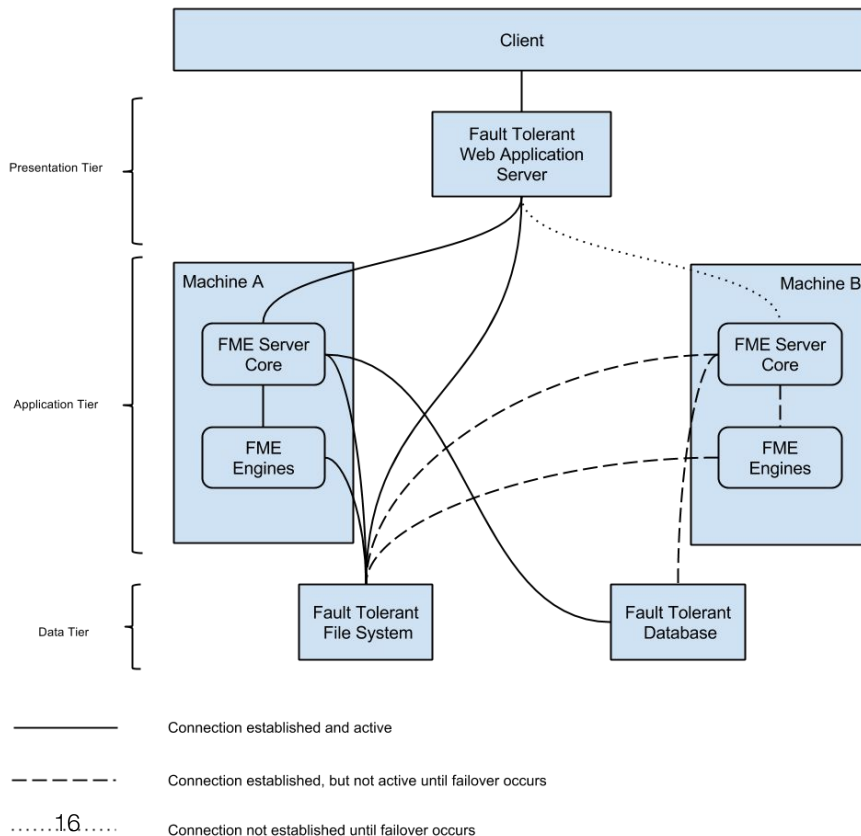
Active-Passive (Active-Standby)

Что?

- Один узел - активный (primary), принимает запись.
- Остальные пассивны: тянут изменения и готовы выйти на смену.
- В PostgreSQL штатная модель: primary + один-несколько standby через streaming replication. Запись всегда в один узел, реплики могут обслуживать только чтение.
- При отказе primary один из standby проморгается в новый primary: вручную или автоматически (Patroni + etcd).
- Пауза на переключение - RTO.

Плюсы и минусы

- + Плюсы: простота и консистентность, отлично ложится на ACID-нагрузку.
- + Минусы: запись упирается в один узел (вертикальный потолок), мощность реплик частично простаивает, есть кратковременная недоступность записи в момент failover.



Active-Active (Multi-Master)

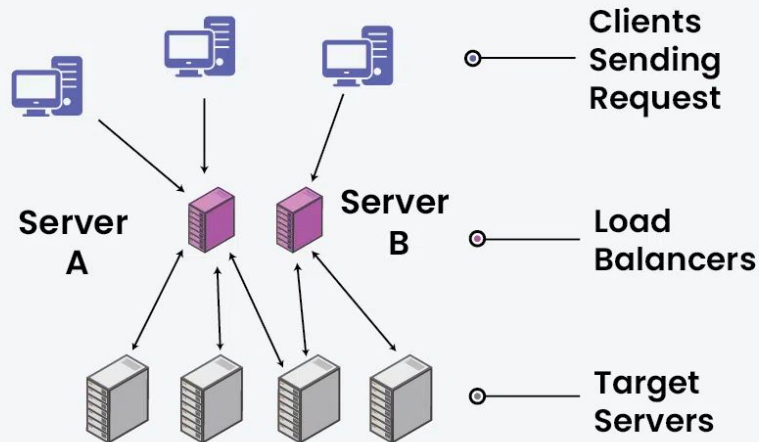
Что?

- Несколько узлов принимают запись одновременно.
- В PostgreSQL это не из коробки: ядро всегда single-primary. Делается внешними решениями: EDB Postgres Distributed (BDR), pgEdge. Citus - это скорее шардинг с координатором, не классический multi-master.

Плюсы и минусы

- + Плюсы: масштабирование записи, гео-распределение (узел рядом с пользователем = низкая задержка), формально нулевое время failover (любой узел рабочий).
- + Минусы: сложность разрешения конфликтов записи: два узла одновременно меняют одну и ту же строку и нужна стратегия разрешения (last-write-wins или кастомная логика) - отказ от строгой консистентности в пользу eventual.

Active Active Architecture



Status: **Server A is online, Server B is online**

Лучшие практики отказоустойчивости

Проектирование

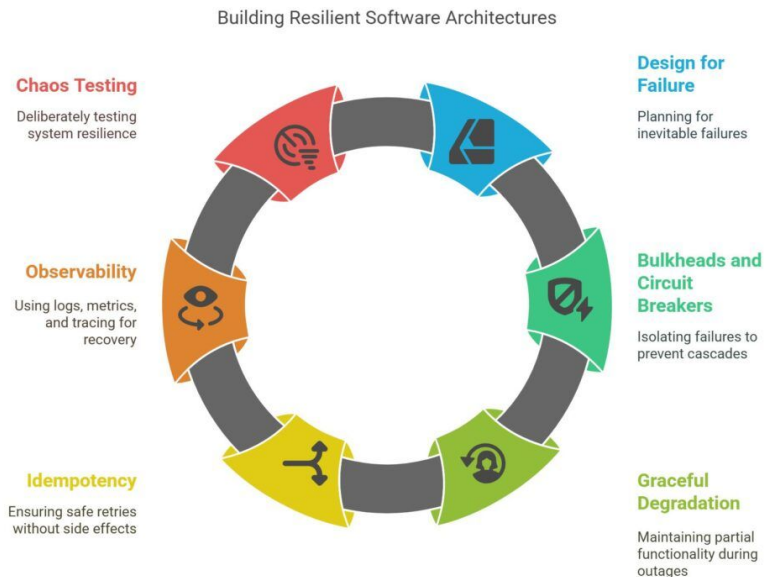
- + При проектировании: design for failure / design for reliability - процесс, благодаря которому система выполняет свои функции в заданной среде в течение ожидаемого срока службы

Разработка

- + При разработке: reliability patterns (паттерны отказоустойчивости)

Тестирование

- + При тестировании: chaos engineering
- + <https://github.com/dastergon/awesome-e-chaos-engineering>





Как обеспечить отказоустойчивость приложения?

Само приложение.
Что можно придумать?

1 экземпляр
Монолит
Отказ = не работает полностью

Приложение



Как обеспечить отказоустойчивость приложения?

Само приложение.
Что можно придумать?

1 экземпляр
Монолит
Отказ = не работает полностью

Приложение

Резерв



3 экземпляра
Резервирование
Отказ одного = не работает частично

Приложение 1

Приложение 2

Приложение 3

Что для этого нужно

- + stateless
- + Балансировщик нагрузки
- + Избыточность (N-копий)
- + healthcheck'и

Следствия

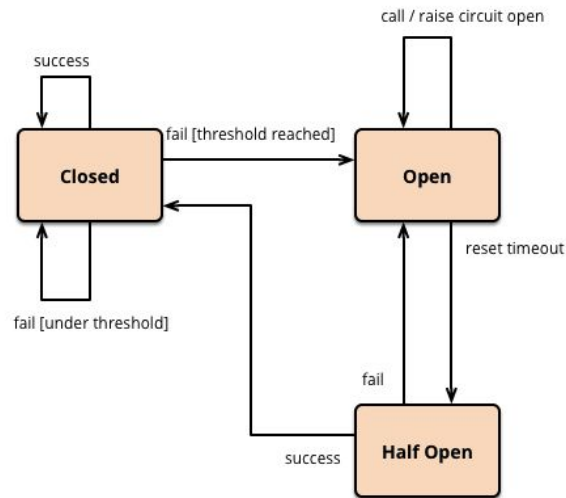
- + Больше платим
- + SPOF переезжает на слой ниже (БД)
- + Интеграции все еще не защищены и не стабильны
- + В монолите сложно частично изолировать отказы

Как обеспечить отказоустойчивость интеграции?

Circuit breaker

Что?

- Суть: перестать вовремя отправлять запросы в упавший/медленный сервис.
- Автомат на три состояния: Closed (пропускаем), Open (сразу отказ, не ходим), Half-Open (пробный запрос: ожил - Closed, нет - снова Open).
- Когда применять: синхронные вызовы к внешней зависимости, особенно если она склонна тормозить под нагрузкой.
- Компромиссы:
 - Open breaker = осознанный отказ части функциональности ради выживания целого
 - Неверные пороги/таймеры дают либо ложные срабатывания, либо запоздалую реакцию



Как обеспечить отказоустойчивость интеграции?

Fallback / Graceful Degradation / Fail Fast

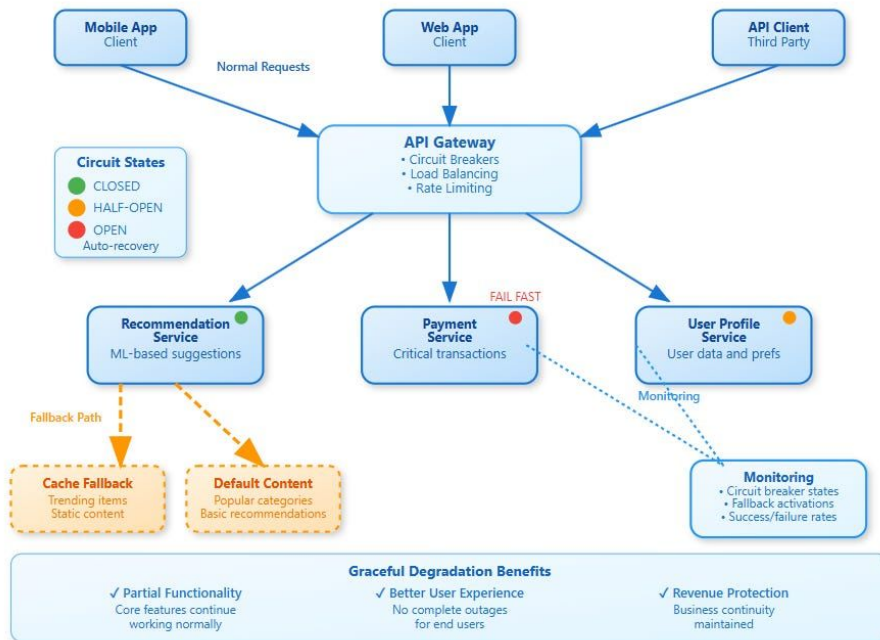
Fallback - Что?

- Суть: вместо ошибки отдать запасной ответ - кэш, дефолтный ответ, упрощенный режим. Дegradiруем часть, а не все приложение.
- Когда применять: есть осмысленный запасной вариант (последнее известное значение из кэша, дефолтная выдача, заглушка) и сценарий терпит неполноту.
- Компромиссы: можно показать устаревшие/неполные данные (риск ввести в заблуждение).

Fail Fast - Что?

- Суть: обнаружить проблему как можно раньше и сразу вернуть явную ошибку. "Лучше громко упасть сейчас, чем тихо сломаться потом".
- Когда применять: на входных границах (валидация запроса, контракта, схемы); при старте сервиса; когда продолжение испортит состояние или впустую сожжет ресурсы.
- Компромиссы: fail fast + агрессивные retry без backoff/jitter = retry storm; слишком строгие пороги/таймауты дают ложные отказы на нормальных всплесках нагрузки.

Graceful Degradation Architecture





Как обеспечить отказоустойчивость интеграции?

Rate Limiter

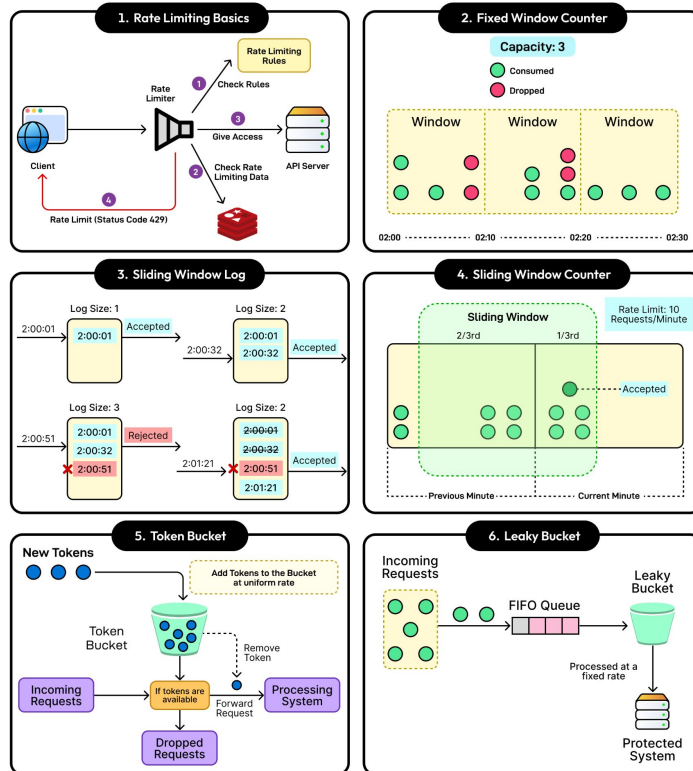
Что?

- Суть: ограничить число запросов на единицу времени от клиента/ключа. Сверх лимита - отказ (обычно 429 Too Many Requests) или придерживание.
- Защищаем емкость системы (Anti-DDOS).
- Иногда применяется для монетизации трафика (привет, лимиты LLM).
- Компромиссы: слишком жесткий лимит режет легитимный трафик; нужен внятный ответ клиенту (429 + Retry-After), иначе он закидает ретраями и усилит нагрузку



A Guide to Rate Limiting Strategies

ByteByteGo





Выводы

- Старайтесь оценивать инженерные решение сквозь призму бюджета
- Быть хорошим инженером нетривиальная задача
- Вы инженеры, а не крudoшлепы
- Надо брать ответственность за свои решения и за спроектированные системы
- Компромиссы повсюду: идеальное враг хорошего
- Нет решений без обоснований
- Часто важны аппроксимации и быстрые расчеты
- Предполагайте, что все точно сломается
- Регулярно снижайте сложность проектируемых приложений
- Не бойтесь нового и постоянно учитесь
- Вас не заменит ИИ, если вы будете адаптироваться
- Теория без практики не работает



НАЦИОНАЛЬНЫЙ ИССЛЕДОВАТЕЛЬСКИЙ
УНИВЕРСИТЕТ